

# Assignment 1

## A Configurable Neural Network from Scratch for Binary Classification

*Forward Pass, Backpropagation, Batch Gradient Descent — Applied to MNIST (Digit 0 vs. Not 0)*

### 1. Objective

In this assignment you will implement a feedforward neural network completely from scratch — no deep learning framework (no PyTorch, TensorFlow, Keras, or scikit-learn's neural network classes) — and use it to solve a real binary classification problem. The network's depth, the width of each layer, and the activation function of each hidden layer must all be configurable by the user; only the output layer is fixed, with exactly one neuron using the sigmoid activation function, since the task is binary classification.

You will first build and validate the generic network engine (forward pass, backpropagation, and batch gradient descent), and then apply it to the MNIST dataset reduced to two classes: “digit is 0” versus “digit is not 0.”

### 2. Configurable Network Specification

The user of your implementation must be able to specify, before training:

- The number of layers  $L$  (excluding the input layer, including the output layer).
- The width  $n_l$  (number of neurons) of every hidden layer  $l = 1, \dots, L-1$ .
- The activation function  $g_l$  used by every hidden layer (e.g. sigmoid, tanh, or ReLU).

The output layer (layer  $L$ ) always has exactly one neuron, and it always uses the sigmoid activation function — this is fixed and not user-configurable, since it is what turns the network into a binary classifier.

A convenient way to expose this configurability is to accept a list of layer sizes and a list of hidden-layer activation names, for example:

```
layer_sizes = [784, 64, 32, 1]      # n0 (input) , n1 , n2 , n3(=1, output)
activations = ['relu', 'tanh']      # one entry per hidden layer (n1, n2); output is
always sigmoid
```

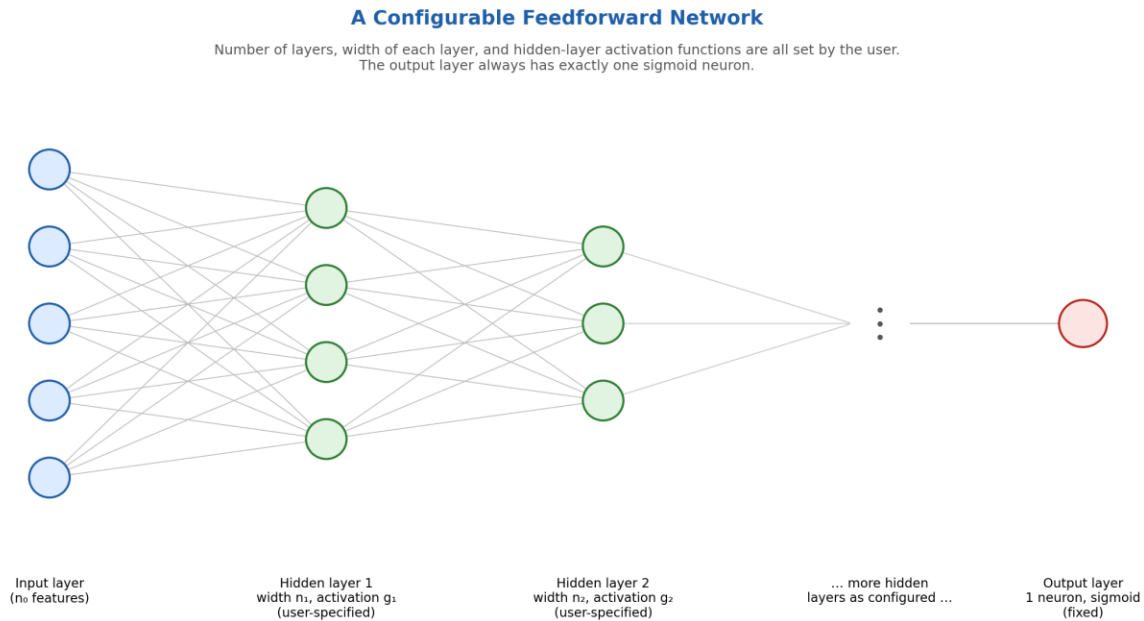


Figure 1: The architecture is entirely configurable except for the fixed single-neuron sigmoid output layer.

### 3. Notation and Forward Pass

Use the following notation consistently throughout your implementation and report (matching what was used in the backpropagation derivation covered in class):

- $z_j^{(l)}$  = the affine (pre-activation) input to neuron  $j$  in layer  $l$
- $y_j^{(l)}$  = the activation (output) of neuron  $j$  in layer  $l$ , with  $y_j^{(0)} = x_j$  (the input features)
- $w_{ij}^{(l)}$  and  $w_{0j}^{(l)}$  = the weight and bias feeding neuron  $j$  of layer  $l$  from neuron  $i$  of layer  $l-1$

#### 3.1 Hidden layers (generic activation)

$$z_j^{(l)} = \sum_i w_{ij}^{(l)} y_i^{(l-1)} + w_{0j}^{(l)} \quad \text{for } l = 1, \dots, L-1$$

$$y_j^{(l)} = g(z_j^{(l)}) \quad \leftarrow g_l \text{ is whichever activation the user selected for layer } l$$

#### 3.2 Output layer (fixed sigmoid, single neuron)

$$z_1^{(L)} = \sum_i w_{i1}^{(L)} y_i^{(L-1)} + w_{01}^{(L)}$$

$$\hat{y} = y_1^{(L)} = \sigma(z_1^{(L)}) \quad \leftarrow \text{always sigmoid, regardless of configuration}$$

#### 3.3 Supported hidden-layer activations

| Activation | $g(z)$                            | $g'(z)$ (in terms of $y = g(z)$ ) |
|------------|-----------------------------------|-----------------------------------|
| sigmoid    | $1 / (1 + e^{-z})$                | $y(1 - y)$                        |
| tanh       | $(e^z - e^{-z}) / (e^z + e^{-z})$ | $1 - y^2$                         |
| ReLU       | $\max(0, z)$                      | 1 if $z > 0$ , else 0             |

Your code should be structured so that adding another activation function later only requires adding its  $g$  and  $g'$  in one place, not modifying the forward/backward pass logic.

## 4. Loss Function

Since there is a single sigmoid output neuron, use binary cross-entropy, averaged over all  $N$  examples in the (batch) training set:

$$L = -(1/N) \sum_n [ t^{(n)} \log \hat{y}^{(n)} + (1 - t^{(n)}) \log(1 - \hat{y}^{(n)}) ]$$

where  $t^{(n)}$  is the true binary label and  $\hat{y}^{(n)}$  the network's predicted probability for training example  $n$ .

## 5. Backward Pass (Backpropagation)

Define the error signal at each neuron's pre-activation,  $\delta_j^{(l)} \equiv \partial L / \partial z_j^{(l)}$ , exactly as derived in class. Because the output layer is always sigmoid combined with binary cross-entropy, the output error simplifies to the familiar shortcut:

$$\delta_j^{(L)} = \hat{y} - t \quad \leftarrow \text{derived from the sigmoid + BCE cancellation shown in class}$$

For every hidden layer, the error is propagated backward using the generic activation derivative  $g'$  for that layer (from the table in Section 3.3):

$$\delta_j^{(l)} = \left( \sum_k \delta_k^{(l+1)} w_{j,k}^{(l+1)} \right) \cdot g'_j(z_j^{(l)}) \quad \text{for } l = 1, \dots, L-1$$

Once every  $\delta$  is known, the weight and bias gradients for every layer are:

$$\partial L / \partial w_{ij}^{(l)} = \delta_j^{(l)} \cdot y_i^{(l-1)}, \quad \partial L / \partial w_{0j}^{(l)} = \delta_j^{(l)}$$

Your implementation must compute every  $\delta$  and every weight/bias gradient using this recursion — you may not use an automatic differentiation library (e.g. autograd, torch.autograd) to obtain gradients.

## 6. Batch Gradient Descent

Train using batch gradient descent: compute the forward pass and the gradients for the entire training set (or the full batch you are given) before each weight update — do not use mini-batches or a single-example (stochastic) update for this assignment. Average the gradient over all  $N$  training examples, then update every weight and bias with learning rate  $\eta$ :

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta \cdot (1/N) \sum_n \partial L^{(n)} / \partial w_{ij}^{(l)}$$

$$w_{0j}^{(l)} \leftarrow w_{0j}^{(l)} - \eta \cdot (1/N) \sum_n \partial L^{(n)} / \partial w_{0j}^{(l)}$$

Repeat this for a chosen number of epochs, and track the loss after each epoch so it can be plotted.

## 7. Task: Part A — Generic Neural Network Engine

Implement a reusable module (a class or a small set of functions) with at least the following:

| Component                            | Required Behaviour                                                                                                                                                                             |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| initialize(layer_sizes, activations) | Accepts the list of layer widths (including input size $n_0$ and output size 1) and the list of hidden-layer activation names, then randomly initializes all weight matrices and bias vectors. |
| forward(X)                           | Computes and stores $z^{(l)}$ and $y^{(l)}$ for every layer, for an entire batch $X$ , ending in $\hat{y}$ .                                                                                   |
| compute_loss(y_hat, t)               | Computes the binary cross-entropy loss averaged over the batch.                                                                                                                                |
| backward(X, t)                       | Computes $\delta$ for every layer using the recursion in Section 5, then every $\partial L / \partial W^{(l)}$ and $\partial L / \partial b^{(l)}$ , averaged over the batch.                  |

|                                      |                                                                                                                                                     |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>update_weights(lr)</code>      | Applies the batch gradient descent update rule from Section 6 to every weight and bias.                                                             |
| <code>train(X, t, epochs, lr)</code> | Runs forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update for the requested number of epochs and returns/stores the loss history. |
| <code>predict(X)</code>              | Runs a forward pass and returns the predicted class (0 or 1) by thresholding $\hat{y}$ at 0.5.                                                      |

## 7.1 Validation before moving on

Before applying your engine to MNIST, validate it on a small synthetic 2D binary classification dataset with at least two different configurations (e.g. a single hidden layer vs. two hidden layers, sigmoid vs. tanh vs. ReLU). Confirm that the training loss decreases over epochs for each configuration, and include the resulting loss curves in your report. This step exists to catch bugs in forward/backward pass before the larger MNIST run.

## 8. Task: Part B — Applying the Network to MNIST (0 vs. Not 0)

- Load the MNIST dataset (60,000 training images, 10,000 test images, each 28×28 grayscale).
- Convert the 10-class labels into a binary label:  $t = 1$  if the digit is 0, and  $t = 0$  for every other digit (1–9).
- Flatten each 28×28 image into a 784-dimensional input vector, and normalize pixel values to the range  $[0, 1]$ .
- Note that the two classes will be imbalanced (roughly 1 in 10 images is a 0) — report accuracy alongside precision, recall, and F1-score, not accuracy alone, since accuracy can be misleading on imbalanced data.
- Choose a network configuration (e.g. 784  $\rightarrow$  64 (ReLU)  $\rightarrow$  32 (tanh)  $\rightarrow$  1 (sigmoid)) using your Part A engine, and train it with batch gradient descent on the training set.
- Plot the training loss versus epoch.
- Evaluate on the held-out test set: report accuracy, precision, recall, F1-score, and a confusion matrix.
- Repeat training with at least one different configuration (a different depth, width, or activation function) and briefly compare the two configurations' final test performance and convergence behaviour.

## 9. Constraints

- Allowed libraries: NumPy for all array/matrix operations; Matplotlib for plotting; a data-loading utility (e.g. `keras.datasets.mnist.load_data()` or an equivalent loader) strictly to obtain the raw MNIST images and labels.
- Not allowed: any neural-network layer, optimizer, or automatic-differentiation functionality from PyTorch, TensorFlow/Keras, JAX, or scikit-learn's neural network classes. The forward pass, backward pass, and gradient descent update must be your own code.
- Your implementation must work for any valid `layer_sizes` / `activations` configuration passed in — do not hard-code the number of layers or their widths inside `forward()`/`backward()`.
- Vectorize across the batch dimension using NumPy matrix operations; avoid explicit Python for-loops over individual training examples in `forward()` and `backward()` (looping over layers is fine).

## 10. What to Submit

- Complete, runnable source code for the Part A engine and the Part B MNIST experiment.

- A short report (2–4 pages) containing the validation loss curves from Section 7.1, the MNIST training loss curve(s), and the evaluation metrics and confusion matrix for each configuration tested.

## 11. Submission Instructions

- Submit a single .zip archive containing your source code and report (PDF).
- Name the archive using the format: RollNumber\_Assignment1.zip
- Submit on GCR before the deadline. Late submissions will be graded according to the course's late-submission policy.